

Chapter 2

Capacitated pickup and delivery vehicle routing

2.1 Introduction



In the capacitated vehicle routing problem with pickups and deliveries, we are given a graph $G=(V, E)$, where each edge e is associated with nonnegative edge cost c_e satisfying the triangle inequality, and each vertex v is associated with a pickup or delivery demand d_v . A vehicle, or a fleet of vehicles with limited capacity k , traverses the edges of G and serves all the pickup and delivery demands. During the traversal, the vehicle should never exceed its capacity. In this chapter, we investigate approximation algorithms for two models of the problem; more specifically, we concentrate on the capacitated Dial-a-Ride problem and the k -delivery traveling salesman problem on paths and in trees.

2.1.1 k -delivery traveling salesman problem

In the k -delivery TSP, we are given an undirected complete graph $G = (V, E)$, where a vertex set $V = V_p \cup V_d$ is partitioned into a set V_p of *pickup vertices* and a set V_d of *delivery vertices* and an edge set E with edge costs satisfying the triangle inequality. Each vertex of V_p is associated with an item, and each vertex of V_d requires one item. A vehicle with capacity k gathers products from pickup vertices, and delivers them to delivery vertices. All the items are identical; an item picked up from a vertex in V_p can be delivered to any vertex of V_d . The objective of the k -delivery TSP is to determine a minimum cost hamiltonian

tour of G subject to the vehicle capacity constraint.

The k -delivery TSP is also called the capacitated pickup and delivery traveling salesman problem (CPDTSP) in [20]. Andrew Lim et al. showed that the problem can be solved optimally in time $O(n^2/\min(k, n))$ on paths in [20], where n is the number of vertices of G . The authors proved in the same paper that this problem is NP-hard in the strong sense even for trees with height  reducing from the 3-partition problem. A 2-approximation algorithm for the k -delivery TSP in trees is ~~later~~ given in [3]. This algorithm follows the rule that the vehicle would continue to pick up (or deliver) items if possible.

The best known approximation ratio 5 for the k -delivery TSP in general graphs is due to Charikar et al. in [49]. Their solution follows the same framework as the 9.5-approximation algorithm proposed by Motwani et al. in [64]. The 9.5-approximation algorithm in [64] is also the first constant factor approximation algorithm for the k -delivery TSP in general graphs. These methods use TSP tours involving a subset of vertices as lower bounds for the k -delivery TSP in general graphs.

2.1.2 Capacitated Dial-a-Ride problem

In the capacitated Dial-a-Ride problem, we are given an undirected complete graph $G = (V, E)$ with edge costs satisfying the triangle inequality, a depot r and a set of jobs, where each job is represented by a source vertex and a destination vertex in G . A vehicle with capacity k is used to serve each job by  carrying the item (of one unit) from its source to its destination. The goal is to compute  a shortest tour which starts from r and also returns to r after serving all the jobs. It is required that at any point of the tour the vehicle should never carry more than k items.

When $k = 1$, the capacitated Dial-a-Ride problem is also called the Stacker-Crane problem. The Stacker-Crane problem on paths can be solved optimally as shown in [46]. Frederickson et al. in  proposed a $\frac{9}{5}$ -approximation algorithm for the Stacker-Crane problem in general graphs.  They improved the ratio to $\frac{5}{4}$ for trees later in [25].

For the capacitated Dial-a-Ride problem with arbitrary vehicle capacity, the best known approximation ratio $O(\sqrt{k} \log n)$ in general graphs is given by Charikar et al. in [49]. This algorithm is in fact for a special structured tree, called *the height balanced tree* in [49]. It is proved in [49] that from an α -approximation for a problem on height balanced trees, we can obtain an $O(\alpha \log n)$ -approximation for the same problem in general graphs, if the objective function of the problem involves only edge costs. For paths, Krumke et al. presented a

3-approximation algorithm in [74]. In this chapter we show that their algorithm contains an error. We also give an approximation algorithm with a ratio strictly less than 2.5, for the capacitated Dial-a-Ride problem on paths.

2.1.3 Our results and solution techniques

In this chapter, we present several algorithms for the k -delivery TSP and the capacitated Dial-a-Ride problem and k -delivery TSP on paths and in trees. Our results are summarized as follows:

(1) For the k -delivery TSP on paths, we obtain an optimal linear time algorithm. This improves the running time $O(n^2/\min(k, n))$ of the algorithm described in [20].

(2) For the capacitated Dial-a-Ride problem on paths, we improve the approximation ratio to 2.5 from 3 as in [74].

(3) For the k -delivery TSP in trees, we improve the approximation ratio to $\frac{5}{3}$. The best known approximation ratio 2 is due to Andrew Lim et al. in [3].

We propose a strategy called *the come back rule* in this chapter for the two models of the capacitated vehicle routing problem with pickups and deliveries on paths and in trees. Our algorithms are all based on this strategy. Under *the come back rule*, the vehicle  could not be allowed to cross a particular edge e if some condition is met. Such condition  varies for different problems. In other words, *the come back rule* states that, the vehicle may not consume its load immediately, but  come back and pick up more items for a better schedule. We give the k -delivery TSP on paths as an example. Assume that the vehicle has a capacity of 100, and the vehicle is moving from left to right and carries 40 items before visiting the next vertex u . If u is a delivery vertex, then the vehicle may still choose to not serve u at this time but to come back and pick up 60 more items before visiting u again. As a consequence, the vehicle may traverse several edges without servicing any jobs, even when it has a large load. This is somewhat contrary to our intuition. However, in this chapter we show that the k -delivery TSP on paths can be solved optimally in linear time by adopting this strategy.

Note that for all the problems we solved, we bound our solutions to the optimum of the *preemptive* version of these problems. In the *preemptive* version of these problems, the vehicle is allowed to drop all or part of its current load temporarily at some intermediate points. The vehicle will  come back and collect the dropped items later. Our algorithms also answer the question  to what extent the optimum of the *non-preemptive* version of

these problems are bounded to that of the *preemptive* version of these problems.

This chapter is organized as follows. In Section 2.2, we define some notations and discuss several lower bounds for the capacitated Dial-a-Ride problem and the k -delivery TSP on paths and in trees. In Section 2.3, we present a linear time optimal algorithm for the k -delivery TSP and use this algorithm as the first concrete example of the *come back rule*. In Section 2.4, we show how to utilize the *come back rule* to improve the approximation ratio to 2.5 (from 3 as in [74]) for the capacitated Dial-a-Ride problem on paths. In Section 2.5, we apply the *come back rule* in a more complicated way and improve the approximation ratio to $\frac{5}{3}$ (from 2 as in [3]) for the k -delivery problem in trees. In section 2.6, we show that by following the *come back rule*, we can slightly reduce the approximation ratio for the capacitated Dial-a-Ride problem on height balanced trees, from $8\sqrt{k}$ as in [49] to $6\sqrt{k}$.

2.2 Lower bounds

In this section, we introduce three lower bounds, called the *flow bound*, the *Steiner tree bound* and the *TSP bound*, for the k -delivery TSP and the capacitated Dial-a-Ride problem in trees. These lower bounds are used widely to design exact or approximate solutions for these problems. We also define some notations for later discussion. Some notations for the *flow bound* and the *Steiner tree bound* are from [49] and [20].

The *flow bound* for the capacitated Dial-a-Ride problem is defined as follows. Consider an edge $e = (u, v)$ of the path or the tree. Let $f_1(e)$ and $f_2(e)$ be the number of jobs that need to cross e from u to v and from v to u respectively. An observation is that in any transportation, the number of times the vehicle moves from u to v equals the number of times the vehicle moves from v to u . Let $\lambda_e = \max\{f_1(e), f_2(e)\}$. As the vehicle has capacity k , in the optimal solution, e is crossed at least $2\lceil \frac{\lambda_e}{k} \rceil$ times. We refer to this as the *flow bound* for the capacitated Dial-a-Ride problem.

The vehicle is required to visit all the jobs in a single tour, therefore the jobs should be ~~at least~~ connected in some way. The *Steiner tree bound* for the capacitated Dial-a-Ride problem considers the cost of connecting the jobs. The formal definition of the *Steiner tree bound* is as follows. A vertex is called *interest* if it lies on a path to serve a job. Let T be the Steiner tree connecting all such vertices. Then the *Steiner tree bound* states that any Dial-a-Ride tour must traverse every edge in T at least twice. We give the capacitated Dial-a-Ride problem on paths as an example. Assume in an instance we only have two jobs

j_1 and j_2 defined on a path and the two jobs are non-overlapping, in the sense that the two path segments P_1 and P_2 obtained from servicing the two jobs are disjoint. Then after servicing one job, say j_1 , the vehicle needs to be forwarded to the source of j_2 to serve j_2 , otherwise the tour is not feasible. Therefore the edges connecting P_1 and P_2 will also have to be traversed twice in any transportation.

The *flow bound* for the k -delivery TSP is defined similarly as that for the capacitated Dial-a-Ride problem. Denote a subtree rooted at vertex u by T_u and the parent of u by $p(u)$. For each vertex v of G , we associate it with a label $a(v)$, which is set to 1 if v is a pickup vertex, and -1 if v is a delivery vertex. For paths, we define $n(v) = \sum_{t=1}^v a(t)$ to be the net number of items among vertices to the left of v (including v); for trees, we define $n(v)$ to be the net number of items of all the vertices in T_v . Then any k -delivery TSP tour must traverse e at least $2\max\{\lceil \frac{|n(v)|}{k} \rceil, 1\}$ times.

The *Steiner tree bound* does not apply to the k -delivery TSP. A lower bound called the *TSP bound* exists for the k -delivery TSP in general graphs. The *TSP bound* simply uses the minimum-cost TSP tour involving a subset of vertices of the graph as a lower bound for the optimum. The 9.5-approximation algorithm in [64] and the 5-approximation algorithm in [49] are based on the *TSP bound*. There are no lower bounds known for the capacitated Dial-a-Ride problem in general graphs.

2.3 A linear time algorithm for the k -delivery TSP on paths

In this section, we propose a simple linear time algorithm for the k -delivery TSP on paths. It is also the first example of our correct rule. As implied by the *flow bound*, an optimal solution of the k -delivery TSP on a path may contain $O(n^2/\min(k, n))$ edges. That is the reason why the algorithm in [20] takes time $O(n^2/\min(k, n))$, which is also the best possible if constructing the final solution by explicitly enumerating its edge sequences. However, in this chapter we show that there is an optimal solution with a succinct representation, whose size is only linear to the number of vertices; in this way we are able to give an optimal algorithm for the k -delivery TSP on paths. The following figure shows such a representation.

In a succinct representation, the optimal solution comprises several subroutes, $r_1, r_2, \dots, r_t$, where $1 \leq t \leq n$. Each vertex is marked to be picked up or delivered by one of such subroutes. For each subroute r_i , where $1 \leq i \leq t$, the vehicle travels along the path from the left most vertex to the right most vertex, which are marked by this subroute. In

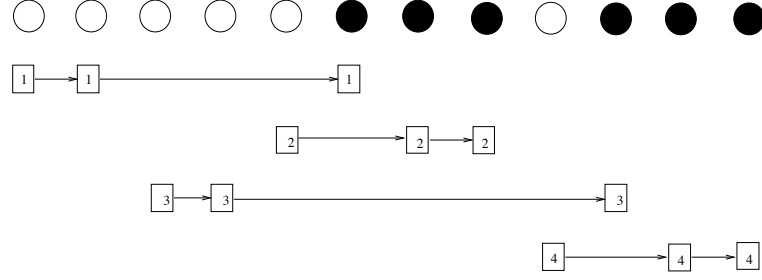


Figure 2.1: A succinct representation of an optimal solution for the k -delivery TSP on a path where $k=2$. A white circle represents a pickup vertex and a black circle represents a delivery vertex.

the meantime, the vehicle services all the vertices marked by the subtour. After reaching the **right most** vertex, the vehicle comes back to the **left most** vertex marked by the next subtour r_{i+1} , and the above process continues. It is clear that such a representation needs only $O(n)$ space.

Procedure come_back1($G, start, direction$)

Input: An undirected graph $G = (V, E)$ defined on a path with nonnegative edge costs; $start$ is the starting point of the path segment; the algorithm will process leftwards if $direction = 1$, and rightwards if $direction = -1$

Output: A succinct representation of an optimal solution for the path segment

```

1  Comment: initialization

2  init_stack(vehicle)

3  init_stack(repository)

4  route_number=i=0

5

6  Comment: mark each vertex

7  while  $0 \leq i \leq |V| - 1$ 

8      if the  $i$ th vertex is a pickup vertex
    
```

```

9      Comment: the vehicle carries less than k unit products
10     if vehicle.size() < k then
11         vehicle.push(i)
12         mark[i]=route_number
13     else
14         repository.push(i)
15     endif
16     else if the ith vertex is a delivery vertex then
17         if  $k - \text{vehicle.size()} = \text{repository.size()} \bmod k$  then
18              $m = \text{repository.size()} \bmod k$ 
19             if  $m=0$  then
20                  $m=k$ 
21             endif
22             route_number=route_number+1
23             for  $j=1$  to  $m$ 
24                 vehicle.push(repository.pop())
25                 mark[i]=route_number
26             endfor
27         endif
28          $v = \text{vehicle.pop}()$ 
29         mark[i]=route_number
30         delivery[i]=v

```

```

31     endif
32      $i = i + \text{direction}$ 
33 endwhile
34 return mark, delivery

```

Figure 2.2: The algorithm for the k -delivery TSP on a path segment

Our main algorithm, called `come_back1` is for a path segment where $A(v)$ is 0 for the right most vertex v , and greater than 0 for every other vertex on this segment. Its pseudo code is listed in Figure 2.2. The algorithm simply scans the segment from left to right, meanwhile it maintains a route number and two stacks. The route number represents the number of the current subtour, and is used to mark each vertex for a succinct representation. A stack called *vehicle* is used to simulate the behaviour of the vehicle. Pushing a vertex to the stack has the same effect as loading one item into the vehicle, and popping from the stack means that the vehicle delivers one item. In the algorithm, a pickup vertex is put into this stack if the current vehicle load is less than k . Another stack called *repository* stores all the pickup vertices which can be served when the vehicle passes by (the vehicle is full). In the procedure, a pickup vertex is put into the *vehicle* stack if the current vehicle load is less than k . Otherwise it is stored in another stack called *repository*. Note that the vertices in the *vehicle* stack belong to the current subtour, and the vertices in the *repository* stack are in later subtours.

The *come back* rule applies when a particular delivery vertex u is to be visited. For a particular time t , we define $n_1(t)$ and $n_2(t)$ to be the number of items in the vehicle and that in the repository at time t respectively. Assume the vehicle is moving rightwards and at time t' it reaches which is the left end of an edge $e = (u, v)$. For the k -delivery TSP on paths, the triggering condition of the *come back* rule is whether $(n_1(t') + n_2(t')) \bmod k = 0$. In other words, the vehicle is allowed to cross e from u to v only if $(n_1(t') + n_2(t')) \bmod k \neq 0$. Otherwise, the current subtour is terminated and the vehicle comes back to pick up the topmost $(n_2(t') \bmod k)$ items in the repository. This can be implemented by increasing the route number by 1 and transferring the topmost $n_2(t') \bmod k$ items of the *repository* stack to the *vehicle-completed* stack. To ease the analysis, we assume that the vehicle picks up these items on its way back from u (when the vehicle is moving leftwards).

The **come back** rule guarantees that the solution produced by this algorithm abides the flow bound for the k -delivery TSP. Therefore in this solution each edge e is traversed exactly $2\lceil \frac{n(e)}{k} \rceil$ times. Before proving this claim, we show that the solution may not be optimal if we do not come back when the triggering condition is satisfied. Consider the example in Figure 2.1. Let the first two delivery vertices be u and v in this example. Let the edge connecting u and v be e . Then when the second delivery vertex v is to be visited at time t , one item remains in the vehicle and three items are stored in the repository. Therefore $(n_1(t) + n_2(t)) \bmod 2 = 0$, and according to our rule the vehicle should come back and pick up one more item before serving v . If otherwise we continue to dump the item in the vehicle to v in the first subtour, then e would have to be traversed at least 2 more times from left to right, since there are 3 pickup vertices still not served among the vertices to the left of v . Such a solution is not optimal as e is only allowed to be crossed $2\lceil \frac{n(e)}{2} \rceil = 4$ times.

The following lemma is crucial to prove that the algorithm indeed finds an optimal solution.

Lemma 2.3.1. *For an edge e of the path segment, let the subtours found by the algorithm passing through e be $r_1, r_2, \dots, r_m$. Then in each of the routes $r_3, \dots, r_m$, the vehicle crosses e from left to right with exactly k items. And the vehicle carries the rest (more than k) items in r_1 and r_2 .*

Proof. Let the left and right ends of e be u and v respectively. Notice that when the vehicle crosses e in r_1 , all the delivery vertices to the left of v must have already been serviced. Otherwise, let v' be such a delivery vertex. Then the vehicle must carry nothing when visiting v' , or else the vehicle can deliver one item to v' . But this is either contradictory to the definition of the path segment  work on, or is contradictory to the **come back** rule. Therefore at time t_1 just before the vehicle crosses e from u to v in r_1 , the number of items in the two stacks equals $n(e)$.

Let r be the residual $n(e) \bmod k$. We already know that $n_1(t_1) + n_2(t_1) \bmod k$ equals r . The vehicle would cross e again in r_2 , only if there exists an edge $e' = (u', v')$ such that v' is to the right of both v and u' , and $n_1(t') + n_2(t') \bmod k$ equals 0 where t' is the time just after the vehicle services v' in r_1 . In this case, the vehicle would cross e from v to u to pick up some more items. Let this time (just before the vehicle visits u) be t_2 . Then we have $n_2(t_2) = n_2(t_1)$ and $n_1(t_2) + n_2(t_2) \bmod k = 0$. The first equality holds because in the algorithm the repository is operated as a stack, so at time t_2 the items in $n_2(t_1)$ remain

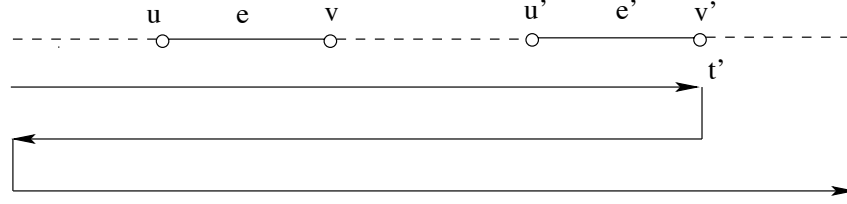


Figure 2.3: The first and second routes.

untouched in $n_2(t_2)$, and the items in $n_2(t')$ but not in $n_2(t_1)$ (collected from some pickup vertices between v and v') must have already been transferred to the vehicle stack. The second equality holds since at time t' all the delivery vertices between v and v' must have already been serviced, so when the vehicle moves straight from u' to u (from time t' to t_2), the only change on the two stacks is to transfer some items from the repository stack to the vehicle stack. Therefore $n_1(t_2) + n_2(t_2) = n_1(t') + n_2(t')$, according to the definition of t' , the equality $n_1(t_2) + n_2(t_2) \bmod k = 0$ holds.

We claim that $n_1(t_2) = n_1(t_1) - r$. According to the definition of r , we have that $n_1(t_1) - r + n_2(t_1) \bmod k = 0$. Since $n_2(t_2) = n_2(t_1)$, we obtain that $n_1(t_1) - r + n_2(t_2) \bmod k = 0$. We know that $n_1(t_2) + n_2(t_2) \bmod k = 0$, and we have that both $n_1(t_1)$ and $n_1(t_2)$ are no more than k , we establish that $n_1(t_2) = n_1(t_1) - r$.

Therefore the vehicle would pick up another $k - n_1(t_1) + r$ items from the left of u before it crosses e again from u to v in r_2 . So in r_2 , the vehicle crosses e from left to right with k items; in r_1 and r_2 , the vehicle crosses e from left to right with $n_1(t_1) + k - n_1(t_1) + r = k + r$ distinct items in total.

Similarly we can prove that the vehicle carries exactly k items when crossing e from left to right in each of the routes $r_3, \dots, r_m$. We show the proof for r_3 . Again the vehicle would cross e in r_3 , only if there exists an edge $e'' = (u'', v'')$ such that v'' is to the right of both u'' and v' . Let t'' be the time just before the vehicle comes back from u'' , then $n_1(t'') + n_2(t'') \bmod k = 0$. Let t_3 be the time just after the vehicle crosses e from v to u in r_2 (back from u''). Similarly as in the above discussion, we have that $n_1(t'') + n_2(t'') = n_1(t_3) + n_2(t_3)$. Since the vehicle carries exactly $k + r$ items when crossing e from left to right in r_1 and r_2 , we obtain that $n_2(t_3) \bmod k = 0$. With $n_1(t_3) + n_2(t_3) \bmod k = 0$, it follows that $n_1(t_3) \bmod k = 0$. The vehicle chooses to cross e from right to left for more items at time t_3 ,

therefore the vehicle must be empty. The vehicle would pick up k items from the repository and cross e from left to right with k items in r_3 .

□

The following theorem can be obtained by directly applying Lemma 2.3.1.

Theorem 2.3.2. *For an edge $e = (v, v + 1)$ of the segment, the tour found by the main algorithm passes through e exactly $2\lceil \frac{n(e)}{k} \rceil$ times.*

Note that the vehicle may carry more than $(n(e) \bmod k)$ but less than k items when crossing an edge e from left to right in the first subtour. This is due to the fact that the priority of loading items is given to the vehicle stack. However the algorithm is still optimal as we proved in Lemma 2.3.1 at the vehicle carries $k + (n(e) \bmod k)$ distinct items in total when crossing e in the first and second subtours from left to right.

Next we complete the final algorithm for the k -delivery TSP on a path. Our strategy is similar to that in [20] for the 1-delivery TSP on a path. The algorithm scans the path from left to right, and uses the procedure `come_back1` to serve the segments whose first vertex is a pickup vertex, and `go` through other segments. The minimal segments whose net number of items is 0 can be found at the same time. When it reaches the right most vertex of the path, it scans the path from right to left, and serves the `rest` segments. For the sake of completeness, we list the pseudo code in the following figure:

2.4 Approximation algorithms for the capacitated Dial-a-Ride problem on paths

In this section we show how to utilize the `come back` rule to improve the approximation ratio to 2.5 for the capacitated Dial-a-Ride problem with arbitrary capacity on paths. Our algorithm first decomposes the original problem into independent subproblems, then the `come back` rule is applied for the subproblems. We also show that the best known 3-approximation algorithm (called the KRW algorithm in the sequel) for the capacitated Dial-a-Ride problem on paths in [74] is flawed. This error can be corrected by applying the main idea within our decomposition framework. The approximation ratio 2.5 is obtained by balancing the two solutions produced by our algorithm and the KRW algorithm fused with our decomposition strategy.

Input:  undirected graph $G = (V, E)$ defined on a path, with nonnegative edge costs
Output: A succinct representation of an optimal solution

```

1 for  $i=0$  to  $|V| - 1$ 
2     if the  $i$ th vertex is a pickup vertex
3         call come_back1( $G, i, 1$ )
4     else if the  $i$ th vertex is a delivery vertex then
5         forward the vehicle to the next segment
6     endif
7 endfor
8
9 for  $i = |V| - 1$  to 0
10    if the  $i$ th vertex is a pickup vertex
11        call come_back1( $G, i, -1$ )
12    else if the  $i$ th vertex is a delivery vertex then
13        forward the vehicle to the next segment
14    endif
15 endfor

```

Figure 2.4: The algorithm for the k -delivery TSP on a path

2.4.1 A decomposition strategy for the capacitated Dial-a-Ride problem on paths

Our algorithm is iterative. In each iteration, the algorithm deals with a smaller size independent subproblem. The subproblem is formed by selecting a suitable subset of jobs (arcs) from the original graph. These jobs are removed from the graph once the subproblem is solved. We show an example of the decomposition in Figure 2.5.

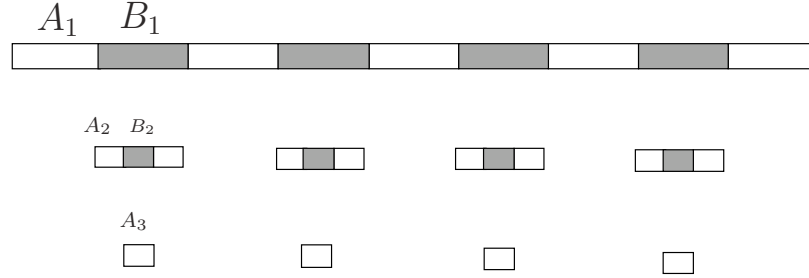


Figure 2.5: A decomposition of a capacitated Dial-a-Ride instance.

Let $J^{\rightarrow}$ be the set of jobs whose sources are to the left of their destinations respectively. Similarly, let $J^{\leftarrow}$ be the set of jobs whose sources are to the right of their destinations respectively. Our algorithm deals with the jobs in $J^{\rightarrow}$ and $J^{\leftarrow}$ separately. In Figure 2.5, it is assumed that all the jobs are from $J^{\rightarrow}$. The algorithm and the analysis can be easily adapted for the jobs in $J^{\leftarrow}$.

We illustrate the first two rounds of the decomposition in Figure 2.5. We say a job j crosses an edge e if and only if e is on the path to service j . Defining an A -segment (B -segment) to be a maximal set of contiguous edges crossed by at most (more than) k jobs, then the path can be viewed as a sequence of alternating A -segments and B -segments. In the first round of our algorithm, a set S of jobs are first selected by applying a job selection procedure on the original problem instance. S includes all the jobs crossing some edges of the A -segments, and possibly also some jobs which do not cross any edges of the A -segments. More details of the job selection procedure are illustrated later in Figure 2.6. After determining S , we remark the A -segments and B -segments of the path with respect to the jobs in S . In Figure 2.5, A_1 denotes all such A -segments and B_1 denotes all such B -segments. A feasible schedule is then calculated to service the jobs in S based on the

partition of the segments. The procedure to calculate the schedule is described in Figure 2.7. In the second round, only the segments in B_1 will be considered, and the set A_2 and B_2 are formed similarly. In general, if letting $\lambda = \max_e(\lambda_e)$ (recall that λ_e is the maximum number of jobs crossing e in one direction), and denoting the total edge cost in a segment S by $\|S\|$, then there are λ such iterations, and we have that $\|A_1\| + \|B_1\| = L$ (L is the total edge cost of the path), and $\|A_i\| + \|B_i\| = \|B_{i-1}\|$, for $2 \leq i \leq \lambda$.

We use the flow bound $2 \sum_e \lambda_e c_e$ to approximate the capacitated Dial-a-Ride problem on paths. In iteration i , only a subset S of the jobs is selected to form the subproblem for this iteration. The set S includes all the jobs which cross some A -segments, and some additional jobs lie within the B -segments. The purpose of including these additional jobs is to ensure that, after the removal of these jobs from the graph in the end of each iteration, the number of crossings of the remaining jobs over a particular edge in a B -segment decreases by at least k . This guarantees that we can use $\|A_i\| + \|B_i\|$ as the lower bound to service these jobs. This also allows us to focus on designing approximation algorithms for the subproblem represented by S . Assume we have an α -approximation algorithm for each subproblem, namely it finds a transportation with cost at most $\alpha \cdot (\|A_i\| + \|B_i\|)$, then it is easy to see that the total cost of solving all the subproblems is bounded by $\alpha \cdot \sum_e \lambda_e c_e$.

The procedure in Figure 2.6 shows the details of selecting the jobs for each iteration of our algorithm.

The job selection procedure is for a particular B -segment B_i . It firstly put all the jobs crossing some A -segments of B_i (with respect to all the remaining jobs) to S . Then we check whether there is an edge e in a B -segment of B_i (with respect to all the remaining jobs) which is crossed by $k_1 < k$ jobs in S . If such an edge e exists, then we arbitrarily choose $k - k_1$ jobs crossing e . These jobs must have their two ends in the same B -segment.

The following simple lemma is guaranteed by steps 10-12 of the job selection procedure. It is crucial for designing approximation algorithms for the subproblems. After remarking the A -segments and B -segments, it is equivalent to say that every selected job must cross an edge of some A -segment.

Lemma 2.4.1. *In each subproblem, every selected job must cross at least one edge which needs to be traversed by at most k selected jobs.*

Finally we need to show that our decomposition strategy is correct. We propose the following lemma without proof since it is directly implied by the selection procedure.

Procedure `select_edges( $G, B_i$ )`

Input: an undirected graph $G = (V, E)$ defined on a path, with nonnegative edge costs, and a B -segment B_i

Output: a set S of jobs

```

1   $J_A \leftarrow$  all the jobs crossing some  $A$ -segments of  $B_i$ 
2   $J_B \leftarrow$  all the jobs not in  $J_A$ 
3
4   $S \leftarrow J_A$ 
5  while  $\exists e \in B_{i+1}$  s.t.  $e$  is crossed by  $k_1 < k$  jobs in  $S$  do
6       $S \leftarrow S + \{ k - k_1 \text{ jobs crossing } e \text{ in } J_B \}$ 
7       $J_B \leftarrow J_B - \{ \text{the } k - k_1 \text{ jobs selected in step 6} \}$ 
8  endwhile
9
10 while  $\exists j \in S$  s.t. every edge crossed by  $j$  is traversed by  $\geq k$  jobs in  $S$  do
11      $S \leftarrow S - \{ j \}$ 
12 endwhile
13
14 remark the  $A$ -segments and  $B$ -segments for  $S$  in  $B_{i+1}$ 
```

Figure 2.6: A procedure for selecting a subset of jobs to form a subproblem

Lemma 2.4.2. *The number of crossings of the remaining jobs on an edge e in iteration i , is at most $\lambda_e - i + 1$.*

2.4.2 The **come back** rule for the capacitated Dial-a-Ride **problem** on paths

With the clearly defined subproblems, we are ready to present an approximation algorithm for the capacitated Dial-a-Ride problem on paths. The input to the algorithm is a subproblem which can be viewed as a sequence of alternating A -segments and B -segments (starting and ending with A -segments). Let $\|A\|$ and $\|B\|$ denote the total edge cost of all the A -segments and B -segments respectively, then algorithm `come_back2` outputs a transportation with cost bounded by $\|A\| + 3\|B\|$.

Algorithm `come_back2`(G, S)

Input: an undirected graph $G = (V, E)$ defined on a path, with nonnegative edge costs, and a sequence S of alternating A -segments and B -segments

Output: a transportation servicing all the jobs in S with cost at most $|A| + 3|B|$

```

1  for each segment  $S_i$  in  $S$ 
2      if  $S_i$  is an  $A$ -segment then
3          pick up all the jobs originating from  $S_i$ 
4      else
5          service all the jobs ending at  $S_i$ 
6          go to the starting vertex of  $S_i$ 
7          pick up all the jobs originating from  $S_i$ 
8      endif
9  endfor
    
```

Figure 2.7: The algorithm for solving a subproblem for the capacitated Dial-a-Ride **problem**

The algorithm in Figure 2.7 adopts the come back rule. The triggering condition is that whether the ending (i.e., last) point of a B -segment is reached for the first time. So when the current segment S is an A -segment, the vehicle would pick up any new jobs it

encountered in S and also deliver some of its load to S if these jobs end in S . When S is a B -segment, the vehicle would first service all the jobs ending in S till the ending point of S is reached. Then the vehicle would come back to the starting point of S and begin to service all the new jobs originated from S . An example of the transportation found by the algorithm is given in Figure 2.8.

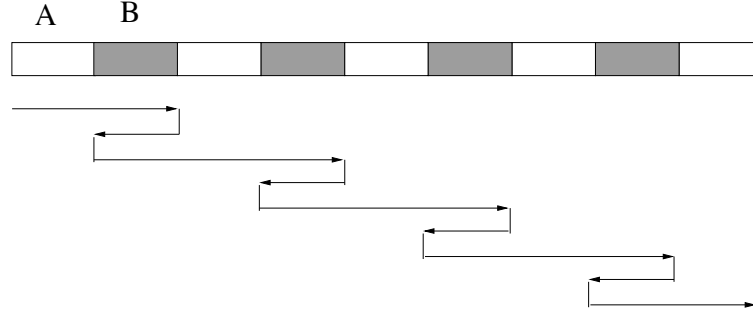


Figure 2.8: A capacitated Dial-a-Ride example of the **come back** strategy

It is critical to show that the come back strategy produces a feasible transportation, namely the vehicle should never carry more than k items at any point of the route. We prove the correctness of the come back strategy in Lemma 2.4.3.

Lemma 2.4.3. *The transportation produced by algorithm `come_back2` is feasible.*

Proof. Assume the vehicle is passing through a segment S_i . According to the algorithm, all the jobs end prior to the starting vertex of S_i should have already been serviced. Then the lemma holds if S_i is an A -segment, for the number of jobs originate in or before S_A and cross some edges of S_A should be no more than k . This is due to the definition of an A -segment. If S_i is a B -segment, then the next segment S_{i+1} must be an A -segment. Since the vehicle first services all the jobs in S_i , the number of items in the vehicle before collecting the new jobs in S_i , plus the number of new jobs originate from S_i , is at most k . This is because these jobs must cross some edges of S_{i+1} , and according to the definition of an A -segment, the number of such jobs should be less than or equal to k . □

The algorithm `come_back2` can also be adapted easily for the jobs in $J^{\leftarrow}$. After solving each subproblem by the algorithm, we obtain two sets of routes $R^{\rightarrow}$ and $R^{\leftarrow}$, where $R^{\rightarrow}$

represents the set of routes for the jobs in $J^{\rightarrow}$, and $R^{\leftarrow}$ represents the set of routes for the jobs in $J^{\leftarrow}$. We treat each route in $R^{\rightarrow}$ and $R^{\leftarrow}$ as a new job with source and destination the starting and ending vertices of the route respectively, and use the optimal algorithm in [46] for the capacitated Dial-a-Ride problem on paths with unit vehicle capacity, to find another set of edges to connect these routes and form the final solution.

For the sake of completeness, we briefly explain the algorithm in [46]. Firstly a set of balancing directed edges are added for each path edge $e = (u, v)$, to make the number of directed edges from u to v equal to the number of directed edges from v to u . This is from the observation that in the optimal solution, if the vehicle traverses e from one direction, then it needs to visit e again from the other direction. The graph then becomes Eulerian, and the Euler tour of a strongly connected component yields an optimal transportation for the involved jobs. The second step of the algorithm is to connect these strongly connected components, by forming a new graph with the strongly connected components as its vertices, and the cost of the edge between two vertices corresponding to strongly connected components c_1 and c_2 , equals to the minimum distance between a vertex of c_1 and another vertex of c_2 . It is not difficult to see that a minimum spanning tree of the new graph represents a set of path edges connect the strongly connected components with the minimum cost.

The same algorithm can be applied to connect the routes in $R^{\rightarrow}$ and $R^{\leftarrow}$. According to our decomposition strategy, the number of crossings on a particular edge e by these routes, is at most the number of crossings on e in the optimal solution. Thus some balancing directed edges can be added to form strongly connected components among these routes. Let $\|A_{\lambda}\| = \max(\|A_{\lambda}^{\rightarrow}\|, \|A_{\lambda}^{\leftarrow}\|)$, then it is easy to see that the cost to connect all the routes is at most $2(L - \|A_{\lambda}\|)$ (recall that L is the total edge cost of the original path). Since the solution for the subproblems in an iteration under the decomposition strategy has a cost of at most $\|A_i\| + 3\|B_i\|$, the cost of the final solution of this algorithm is bounded by $\sum_{i=1}^{\lambda-1} (\|A_i^{\rightarrow}\| + 3\|B_i^{\rightarrow}\| + \|A_i^{\leftarrow}\| + 3\|B_i^{\leftarrow}\|) + \|A_{\lambda}^{\rightarrow}\| + \|A_{\lambda}^{\leftarrow}\| + (2L - 2\|A_{\lambda}\|)$. As implied in the proof of Theorem 2.4.5, this algorithm alone is a 3-approximation for the capacitated Dial-a-Ride problem on paths.

2.4.3 The KRW algorithm

We improve the ratio to 2.5 by balancing the solutions produced by our algorithm, and the KRW algorithm in [74]. In the following we sketch their algorithm and show that the ratio 3-~~they~~ claimed in [74] can only be justified by combining their main idea with our

decomposition strategy.

For the jobs in $J^{\rightarrow}$, the KRW algorithm first finds a set of transportation segments where each segment consists of several non-overlapping jobs. Thus a vehicle with unit capacity can service the jobs of one such segment by proceeding forward along the segment. We denote such a segment by an unit-capacity segment. The algorithm then groups every k unit-capacity segments to form a set $R^{\rightarrow}$ of new transportation segments, where each new segment can be serviced by forwarding a vehicle with capacity k . The segment set $R^{\leftarrow}$ for the jobs in $J^{\leftarrow}$ is formed similarly. Treating each segment in $R^{\rightarrow}$ and $R^{\leftarrow}$ as a new job, the problem can be transformed to the capacitated Dial-a-Ride problem on paths with unit capacity. The second step of the algorithm in [74] is to use the algorithm in [46] to find a set of augmenting edges with minimum cost to paste the segments together. For each vertex v , define $v+1$ to be its right neighbor. Since the set of arcs $\{(v, v+1), (v+1, v) : 1 \leq v \leq n-1\}$ is a feasible set of connecting arcs, it is easy to see that the augmenting edges found in the second step are bounded by the optimal solution.

To obtain a 3-approximation, the cost of the edges in the segments needs to be bounded by twice of the optimum. The main idea of [74] is to find two unit-capacity segments at a time, where each involved path edge is covered by at least one of the two segments. For a subset of jobs $J \subseteq J^{\rightarrow}$, let $\alpha(J)$ and $\omega(J)$ to be the left- and right-most vertices involved in $J^{\rightarrow}$ respectively. Note that $\alpha(J)$ must be the source of a job, and $\omega(J)$ must be the destination of the same or another job. Let S_1 and S_2 be the two unit-capacity segments the algorithm finds in one iteration. First the job j_1 with $\alpha(J^{\rightarrow})$ as its source vertex is included in S_1 . Then from the jobs whose source vertices lie between the source and destination of j_1 , the job j_2 with the right-most destination is chosen to be in S_2 . Similarly among all the jobs whose source vertices lie between the destinations of j_1 and j_2 , the job j_3 with the right-most destination is chosen to be included in S_1 . By repeating such a process, the two segments are assigned a job at a time alternately until $\omega(J^{\rightarrow})$ is reached. Then the chosen jobs are removed from $J^{\rightarrow}$, and a new iteration begins if $J^{\rightarrow}$ is still not empty. It is proved in [74] that the above greedy selection strategy is crucial to the performance guarantee. An example of a run of the algorithm is shown in figure 2.9. The right part of the figure shows the six unit-capacity segments constructed by the algorithm, and also the order of selecting the jobs.

In one iteration, the KRW algorithm in [74] always finds two unit capacity segments which together cover all the edges between the leftmost source and rightmost destination

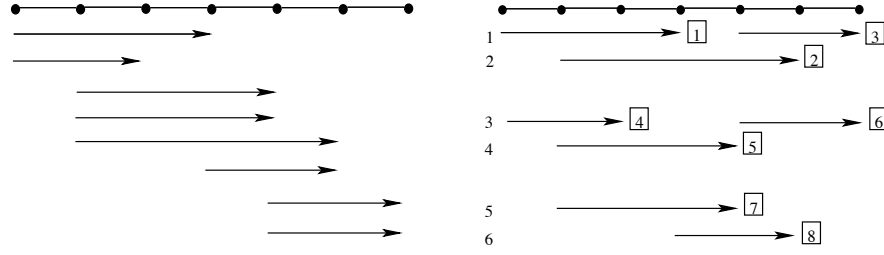


Figure 2.9: The 3-approximation KRW algorithm applied to a graph.

vertices of the remaining jobs. However, there may exist some edges which are not covered by any job. In this case, the KRW algorithm would add some pseudo jobs covering these edges to the original graph. Therefore each unit capacity segment corresponds to a transportation where a vehicle with unit capacity moves straight from the leftmost source to the rightmost destination of the remaining jobs just before this segment is formed. Figure 2.10 shows a case where such a strategy does not apply.

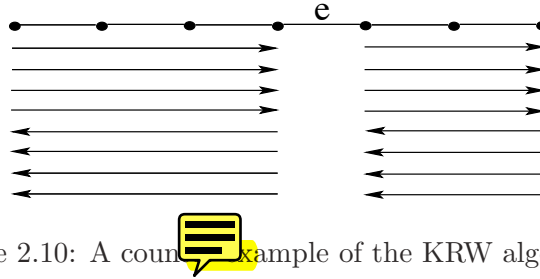


Figure 2.10: A counterexample of the KRW algorithm.

In this example, the cost of the solution produced by the KRW algorithm is unbounded compared to the optimum since the edge e is only traversed twice in the optimal solution, whereas it needs to be traversed once in each of the unit-capacity segments constructed by the KRW algorithm. It is easy to construct similarly structured instances where the number of such segments is arbitrarily large.

Nevertheless, the KRW algorithm can be integrated seamlessly into our decomposition framework. Recall that in each subproblem, a selected job j must cross at least one edge which needs to be traversed by at most k selected jobs. This property allows us to bound the solution cost of the KRW algorithm. More details are given in the proof of Lemma

2.4.4

Lemma 2.4.4. *For a subproblem with A-segment set A_i and B-segment set B_i , the solution given by the KRW algorithm has a cost bounded by $2\|A_i\| + 2\|B_i\|$.*

Proof. Assume $2k$ unit-capacity segments have been created in a run of the KRW algorithm. Let S be the set of jobs crossing a particular edge e . According to the KRW algorithm, e must be covered by every two unit-capacity segments found by the KRW algorithm. Notice that pseudo jobs are added only if all the jobs crossing e have already been consumed. Therefore if $\|S\| \leq k$, all the jobs in S must be serviced within the $2k$ unit-capacity segments. Since in a subproblem a job must cross an edge e , e needs to be traversed by at most k selected jobs, every job is included in the $2k$ unit-capacity segments. This completes the proof. $\square$

2.4.4 Performance analysis

Our final solution is the tour with the smaller cost from the two tours produced by the algorithm `come_back` and the KRW algorithm in [74]. We prove the approximation ratio of this tour is strictly less than 2.5 in Theorem 2.4.5.

Theorem 2.4.5. *The cost of the transportation with smaller cost from the two tours produced by the algorithm `come_back2` and the algorithm in [74], is bounded by 2.5 times the optimum.*

Proof. As mentioned above, the cost of the tour after running algorithm `come_back2`, is at most

$$\sum_{i=1}^{\lambda-1} (\|A_i^{\rightarrow}\| + 3\|B_i^{\rightarrow}\| + \|A_i^{\leftarrow}\| + 3\|B_i^{\leftarrow}\|) + \|A_{\lambda}^{\rightarrow}\| + \|A_{\lambda}^{\leftarrow}\| + (2L - 2\|A_{\lambda}\|) \quad (1)$$

Since $\|A_1^{\rightarrow}\| + \|B_1^{\rightarrow}\| = L$, $\|A_{\lambda}^{\rightarrow}\| = \|B_{\lambda-1}^{\rightarrow}\|$, and $\|A_i^{\rightarrow}\| + \|B_i^{\rightarrow}\| = \|B_{i-1}^{\rightarrow}\|$ for $2 \leq i \leq \lambda - 1$, we have

$$\begin{aligned} \sum_{i=1}^{\lambda-1} (\|A_i^{\rightarrow}\| + 3\|B_i^{\rightarrow}\|) + \|A_{\lambda}^{\rightarrow}\| + (L - \|A_{\lambda}\|) &\leq \sum_{i=1}^{\lambda-1} (2\|A_i^{\rightarrow}\| + 3\|B_i^{\rightarrow}\|) + \|A_{\lambda}^{\rightarrow}\| = \\ &2L + \sum_{i=1}^{\lambda-1} 3\|B_i^{\rightarrow}\| - \|B_{\lambda-1}^{\rightarrow}\| \end{aligned}$$

Similarly,

$$\sum_{i=1}^{\lambda-1} (\|A_i^{\leftarrow}\| + 3\|B_i^{\leftarrow}\|) + \|A_{\lambda}^{\leftarrow}\| + (L - \|A_{\lambda}\|) \leq 2L + \sum_{i=1}^{\lambda-1} 3\|B_i^{\leftarrow}\| - \|B_{\lambda-1}^{\leftarrow}\|$$

Thus (1) is less than or equal to

$$4L + \sum_{i=1}^{\lambda-1} 3\|B_i^{\rightarrow}\| + \sum_{i=1}^{\lambda-1} 3\|B_i^{\leftarrow}\| - \|B_{\lambda-1}^{\rightarrow}\| - \|B_{\lambda-1}^{\leftarrow}\| \quad (2)$$

The cost of the solution by combining the main idea in [74] and our decomposition strategy, is bounded by

$$\sum_{i=1}^{\lambda-1} (2\|A_i^{\rightarrow}\| + 2\|B_i^{\rightarrow}\| + 2\|A_i^{\leftarrow}\| + 2\|B_i^{\leftarrow}\|) + 2\|A_{\lambda}^{\rightarrow}\| + 2\|A_{\lambda}^{\leftarrow}\| + (2L - 2\|A_{\lambda}\|) \quad (3)$$

Eliminating $\|A_i^{\rightarrow}\|$ and $\|A_i^{\leftarrow}\|$ as above, (3) is less than or equal to

$$6L + \sum_{i=1}^{\lambda-1} 2\|B_i^{\rightarrow}\| + \sum_{i=1}^{\lambda-1} 2\|B_i^{\leftarrow}\| - \|B_{\lambda-1}^{\rightarrow}\| - \|B_{\lambda-1}^{\leftarrow}\| \quad (4)$$

The optimal solution has a cost at least twice of

$$\begin{aligned} \|A_1^{\rightarrow}\| + 2\|A_2^{\rightarrow}\| + \cdots + \lambda\|A_{\lambda}^{\rightarrow}\| &= \|A_1^{\rightarrow}\| + 2(\|B_1^{\rightarrow}\| - \|B_2^{\rightarrow}\|) + \cdots + \lambda\|A_{\lambda}^{\rightarrow}\| \\ &= \|A_1^{\rightarrow}\| + 2\|B_1^{\rightarrow}\| + \|B_2^{\rightarrow}\| + \cdots + \|B_{\lambda-1}^{\rightarrow}\| = L + \sum_{i=1}^{\lambda-1} \|B_i^{\rightarrow}\| \end{aligned}$$

Similarly, the optimal solution has a cost at least twice of $L + \sum_{i=1}^{\lambda-1} \|B_i^{\leftarrow}\|$.

Thus we have

$$OPT \geq 2L + \sum_{i=1}^{\lambda-1} \|B_i^{\rightarrow}\| + \sum_{i=1}^{\lambda-1} \|B_i^{\leftarrow}\| \quad (5)$$

Adding (2) and (4), the total cost of the two solutions is at most

$$10L + \sum_{i=1}^{\lambda-1} 5\|B_i^{\rightarrow}\| + \sum_{i=1}^{\lambda-1} 5\|B_i^{\leftarrow}\| - 2\|B_{\lambda-1}^{\rightarrow}\| - 2\|B_{\lambda-1}^{\leftarrow}\| \quad (6)$$

From (5) and (6), the approximation ratio of the final solution is at most

$$\alpha \leq \frac{1}{2} \cdot \frac{10L + \sum_{i=1}^{\lambda-1} 5\|B_i^{\rightarrow}\| + \sum_{i=1}^{\lambda-1} 5\|B_i^{\leftarrow}\| - 2\|B_{\lambda-1}^{\rightarrow}\| - 2\|B_{\lambda-1}^{\leftarrow}\|}{2L + \sum_{i=1}^{\lambda-1} \|B_i^{\rightarrow}\| + \sum_{i=1}^{\lambda-1} \|B_i^{\leftarrow}\|} < 2.5$$

□

2.5 Approximation algorithms for the k -delivery TSP in trees

In this section we study the non-preemptive k -delivery TSP in trees. As shown in [20], the problem is NP-complete even in a tree of height 2. We present a $\frac{5}{3}$ -approximation algorithm called the Half-Load algorithm for the k -delivery TSP in trees of arbitrary heights. This algorithm is a $(\frac{3}{2} - \frac{1}{2k})$ -approximation for the k -delivery TSP in trees of height 2. The Half-Load algorithm is also based on the *come back* rule, albeit in a more complicated form. We first use the $(\frac{3}{2} - \frac{1}{2k})$ -approximation for the k -delivery TSP in trees of height 2 to illustrate our idea.

The following notations are used to describe the two algorithms. We call a branch positive (or negative) if and only if this branch contains a positive (or negative) net number of items. Similarly, a vertex u is positive (or negative) if and only if the subtree T_u is positive (or negative), and an edge $e = (p(u), u)$ is positive (or negative) if and only if u is positive (or negative). For an edge $e = (p(u), u)$, we say some vertices are picked up from (or delivered to) e or u , if and only if we pick up (or deliver) these vertices from (or to) the subtree T_u ; moreover, we say e has a load of $n(e)$, or e contains $n(e)$ vertices, if and only if exactly $n(e)$ net number of items need to be picked up or delivered through e . Finally we denote the *flow bound* for a particular edge e (the number $2\lceil \frac{n(e)}{k} \rceil$) by FB_e . To ease the explanation, we denote $\frac{1}{2} \cdot FB_e$ by LB_e and we say the vehicle visits (crosses, traverses) e if and only if the vehicle moves from $p(u)$ to u .

2.5.1 Exploring the symmetry of the k -Delivery TSP

Our improvements for the k -delivery TSP in trees explore the symmetry inherent in the problem. In the k -delivery TSP, the underlying graph has only two types of vertices. It is not difficult to see that if we flip the type of each vertex, to get a new graph, say G' , then any feasible solution of the k -delivery TSP on G' can be converted to a feasible solution of G with the same edge cost, by just reversing its edge directions. Therefore we have the following lemma.

Lemma 2.5.1. *Any feasible solution of the k -delivery TSP on G' can be converted to a feasible solution of G with the same edge cost, by reversing its edge directions.*

This property allows us to design approximation algorithms for the k -delivery TSP in trees in the following way. We partition the edge set E of the graph into two sets S and $E - S$. Consider an approximation algorithm which is particularly “good” to S , in the sense that in the tour produced by the algorithm, each edge e in S and $E - S$ is traversed at most α and β times LB_e respectively, for some $\alpha < \beta$. Then it is easy to see that after flipping the type of each vertex and obtaining the second tour on G' , each edge e would be crossed at most $(\alpha + \beta)LB_e$ times on average. Therefore the approximation ratio is reduced to $\frac{\alpha + \beta}{2}$ from β after running the same algorithm on both G and G' . In the Half-Load algorithm, S is defined to be the set of positive edges of G . For the k -delivery TSP in trees of height 2, this algorithm achieves $\alpha = 1$ and $\beta = 2$. We defer the analysis of the algorithm in trees of general heights in later sections of this chapter.

2.5.2 Preprocessing step of the Half-Load algorithm

The Half-Load algorithm contains two phases, the planning phase and the actual route generating phase. In the planning phase, the original graph is transformed to a multi-graph G'' as follows. Firstly, in G'' all the pickup and delivery demands are assumed to be on the leaves. This can be done easily by adding a pseudo vertex u' for each non-leaf vertex u of G , and attaching an edge between u and u' with zero cost. Secondly, each positive tree edge $e = (p(u), u)$ is split into several pseudo edges as shown in Figure 2.11 and 2.14. These pseudo edges are incident to $p(u)$ and u and their total load is equal to $n(e)$. Intuitively they record the number of items the vehicle would pick up during each of its visits to T_u . In a tree of height 2, an edge $e = (p(u), u)$ is split into $\lceil \frac{n(u)}{k} \rceil$ pseudo edges. Each of the first $\lfloor \frac{n(u)}{k} \rfloor$ edges has load of k . The last pseudo edge of e contains the residual of $n(e) \bmod k$.

In the example in Figure 2.11, the capacity k is set to 8. A pseudo edge of the transformed tree is represented by a dashed line, and a subtree is represented by a triangle with a number showing the net amount of product inside this subtree. Since the first positive tree edge has a load of more than k , it is split into two edges with loads 8 and 4 respectively. Other tree edges do not split, since they are negative or their loads are less than k .

Given a list L of pseudo edges, we use a procedure called *merge* to produce a new set of pseudo edges, where each new edge (except possibly one) has load more than $\frac{k}{2}$. We list the pseudo code in Figure 2.12.

This procedure repeatedly merges two arbitrary pseudo edges with loads $\leq \frac{k}{2}$ if possible. A set S of pseudo edges which are merged together is treated as a single pseudo edge e' ,

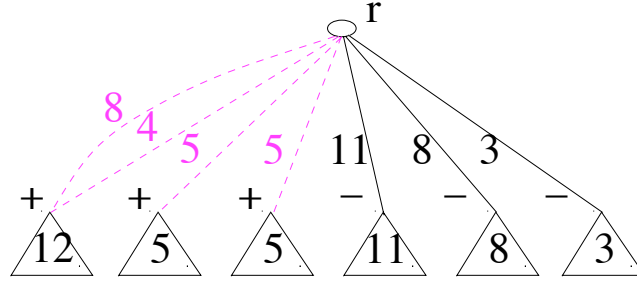


Figure 2.11: an example of building pseudo edges for a vertex u ($k=8$).

procedure merge(L)

Input: list L of pseudo edges

Output: a list of pseudo edges s.t. all except possibly one have loads $> \frac{k}{2}$

```

1 if  $\exists e_i, e_j \in L$  s.t.  $n(e_i), n(e_j) \leq \frac{k}{2}$  then
2      $n(e_i) \leftarrow n(e_i) + n(e_j)$ 
3      $e_i.link \leftarrow$  concatenate the link lists of  $e_i$  and  $e_j$ 
4      $L \leftarrow L - \{e_j\}$ 
5 endif
6
7 return  $L$ 
```

Figure 2.12: the merge procedure for a list of pseudo edges

and when $n(e')$ items are requested from e' , they are actually picked up from the pseudo edges in S . This can be implemented by creating a group link for each group of merged pseudo edges. In addition, each pseudo edge $e = (p(u), u)$ is associated with a children link to remember where its load is from. More specifically, e is linked with a set S of edges which are incident to u and some children of u . In the second phase, when the load of e is requested, these links can be used to locate all the edges that participated in generating e (reachable from e by following the links). We show an example of building pseudo edges in Figure 2.14.

Recall that the task of the planning phase is to split the positive tree edges into positive pseudo edges. The second actual route generating phase is based on these positive pseudo edges. We list the pseudo code of the planning phase in Figure 2.13.

The pseudo edges and their links are built recursively in a bottom-up fashion. Pseudo edges are firstly created from leaves, and then spread to higher levels of the tree. For a positive vertex u , given the list L_+ and L_- which contain all the positive (pseudo) and negative edges from u to its children respectively, the algorithm firstly figures out the internal schedule for the negative edges, by selecting a minimal subset S of edges from L_+ , whose load is just enough to consume the total load of the edges in L_- . A new pseudo edge e' is created from u to its parent and is linked with the edges in S and L_- . The merge procedure is then applied for the rest positive pseudo edges, and each resulting pseudo edge e is propagated one level up by placing a copy e' of e between u and $p(u)$. We also link e' with e through its children link.

The merging is necessary for our $\frac{5}{3}$ -approximation algorithm for the k -delivery TSP in trees of general heights. It also keeps the number of positive pseudo edges in the order of $O(\lceil \frac{n^2}{k} \rceil)$. This ensures that the time and space complexities of the algorithm is $O(\lceil \frac{n^2}{k} \rceil)$. When this phase is finished, for a particular pseudo edge e , we assume that we can trace back all the edges which have participated in generating e . This can be easily done by adding some links between these edges. An example of building L_+ for a positive vertex u is shown in Figure 2.14.

In this example, it is assumed that the pseudo edges from u to its children have already been built. Counting from left to right, the load of the first two pseudo edges is enough to serve the two negative edges. Since only one vertex after consuming the load of the two negative edges, the third positive pseudo edge is linked with the first two positive pseudo edges and the two negative edges to the first pseudo edge e_1 for u . The next two pseudo edges

procedure `half_load_plan`(T_u)

Input: an undirected tree T_u , with nonnegative edge costs

Output: a multi-graph with positive pseudo edges

```

1   $L_+ \leftarrow \emptyset$ 
2  if  $u$  is a leaf then
3      create a new pseudo edge  $e$  from  $u$  to its parent with  $n(e) = 1$ 
4      return  $e$ 
5  endif
6
7  for each positive child  $c$  of  $u$ 
8       $L_+ \leftarrow \text{merge}(L_+ + \text{half\_load\_plan}(T_c))$ 
9  endfor
10
11  $L_- \leftarrow$  all the negative tree edges from  $u$  to its children
12
13  $e' \leftarrow$  a new edge from  $u$  to  $p(u)$ , which is linked with the edges of  $L_-$  and several
    edges of  $L_+$  whose load is just enough to serve the edges of  $L_-$ 
14
15
16 remove these edges from  $L_+$ 
17
18 while  $L_+$  is not empty do
19     create a new edge  $e'$  from  $u$  to its parent
20      $e' \leftarrow$  a copy of the head  $e$  of  $L_+$ 
21      $L_+ \leftarrow L_+ - \{e\}$ 
22 endwhile
23
24  $L_+ \leftarrow$  all the newly created pseudo edges
25 return  $\text{merge}(L_+)$ 

```

Figure 2.13: The planning phase of the Half-Load algorithm for the k -delivery TSP on a tree of general height

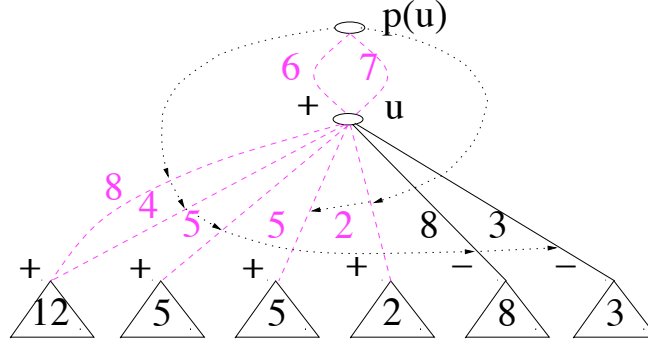


Figure 2.14: An example of building pseudo edges in the planning phase($k=8$).

are merged together to form the second pseudo edge e_2 for u with load 7. This merging is in the sense that the three pseudo edges are linked together to allow the later tracing back of the 7 vertices.

2.5.3 Come back rule for the k -delivery TSP in trees

The **come back** rule of the Half-Load algorithm is applied in the scenario when we are given two lists L_+ and L_- and a vertex u , where the negative edges in L_- are from u to its children, and they must be serviced by the items from the positive pseudo edges in L_+ . The pseudo code of the **come back** rule is listed in Figure 2.15. To ease the explanation, we assume that the tree is of height 2. This procedure also works for trees of general heights. The explanations for general cases will be given in later sections of this chapter.

Two procedures **pickup** and **deliver** are called in the algorithm. The details of the two procedures (picking up and delivering through an edge) are described later for trees of general heights. For trees of height 2, picking up from (delivering to) an edge $e = (p(u), u)$ can be implemented by simply picking up (delivering) the predetermined amount (α in line 8 of Figure 2.15) of product from (to) the leaves of the branch. The vertices not going outside the branch with the vehicle can be serviced during the first visit of the vehicle to this branch.

The algorithm runs in iterations, and in each iteration we maintain two variables e_1 and e_2 pointing to the first edges of L_+ and L_- respectively. Recall that under the **come back** rule, the vehicle may not be allowed to cross an edge e if some condition occurs. Th

Procedure come_back3(α, L_+, L_-)

Input: an undirected tree of arbitrary height, with nonnegative edge costs; an initial vehicle load α ; two lists L_+ and L_- contain some positive and negative edges respectively.

Output: a tour satisfying the capacity constraint

```

1  Comment: the vehicle may already have a load
2  while  $L_+$  and  $L_-$  are not empty do
3       $e_1 \leftarrow$  the head of  $L_+$ 
4       $e_2 \leftarrow$  the head of  $L_-$ 
5
6      if  $e_1$  exists and  $\alpha + n(e_1) \leq k$  then
7          Comment: pick up the vertices from  $e_1$ 
8          pickup( $\alpha, e_1$ )
9           $\alpha \leftarrow \alpha + n(e_1)$ 
10          $L_+ \leftarrow L_+ - \{e_1\}$ 
11     else if load  $\geq n(e_2)$  then
12         Comment: consume  $e_2$ 
13         deliver( $L_+, e_2$ )
14          $\alpha \leftarrow \alpha - n(e_2)$ 
15          $L_- \leftarrow L_- - \{e_2\}$ 
16     else
17         Comment: deliver all the vehicle load to  $e_2$ 
18         deliver( $L_+, e_2$ )
19          $\alpha \leftarrow 0$ 
20     endif
21 endwhile
22
23 return the route and  $L_+$  and  $L_-$ 

```

Figure 2.15: **Come back** rule for the k -delivery TSP in trees

triggering condition in this algorithm is whether the load of e_1 fits the capacity of the vehicle. Therefore in each iteration of the algorithm, if the current load of the vehicle plus the load of e_1 is $\leq k$, then the vehicle will come back and pick up the load of e_1 . Otherwise, the vehicle begins to deliver all or part of its current load to the head e_2 of L_- . Whenever the load of e_1 or e_2 is fully consumed, it is removed from L_+ or L_- . The above process continues until L_+ and L_- become empty.

An example is given in Figure 2.16 to show the effectiveness of our **come back** strategy. In this example, $k = 8$, the input to the algorithm is a list L_+ which contains edges $e_1, e_2, \dots, e_5$ with loads 6, 6, 6, 6, 4 respectively, and a list L_- which contains edges $e'_1, e'_2, \dots, e'_4$ with loads 7, 7, 7, 7 respectively. It is not difficult to verify that only e'_1, e'_2 and e'_3 will be traversed twice if following the **come back** rule. If we flip the types of the vertices and run the algorithm in 2.15, then only e_3 and e_4 need to be visited twice. Therefore on average $e_1, e_2, \dots, e_5, e'_1, e'_2, \dots, e'_4$ will be visited $1, 1, \frac{3}{2}, \frac{3}{2}, 1, \frac{3}{2}, \frac{3}{2}, \frac{3}{2}$ respectively. However, if we follow the strategy in [49] and [3] where the vehicle would continue to pick up (or deliver) items if possible (which we call *the Full-Load strategy* in the sequel), then $e_1, e_2, \dots, e_5, e'_1, e'_2, \dots, e'_4$ will be traversed 1, 2, 2, 2, 2, 1, 2, 2, 2 times respectively. If we flip the types of the vertices and apply the Full-Load strategy, then $e_1, e_2, \dots, e_5, e'_1, e'_2, \dots, e'_4$ will be traversed 1, 2, 2, 1, 1, 1, 2 times respectively. It is not difficult to see that the **come back** rule outperforms the Full-Load strategy clearly in this example.

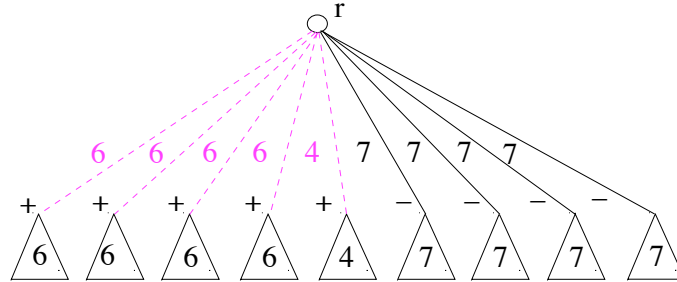


Figure 2.16: An example of the **come back** strategy with $k=8$.

A formal proof of the correctness of our algorithm is listed in the following lemma.

Lemma 2.5.2. *The algorithm in Figure 2.15 approximates the k -delivery TSP in trees of height 2 within $\frac{3}{2}$ of its optimum.*

Proof. It is not difficult to see that the vehicle always obeys the capacity constraint. According to the **come back** rule, each positive edge in L_+ is only visited once. Therefore all we need to show is that after applying the algorithm, each negative edge in L_- is traversed at most twice.

In the algorithm in Figure 2.15, the vehicle starts to deliver its load, if and only if its load plus the load of the next positive edge is more than k . Because L_- consists of real negative tree edges, during every visit on e , this check is enforced. Also note that if the vehicle starts to visit e for the first time, then the subsequent pickup (or delivery) tasks occur also on e , until all the load of e is consumed. Thus during every two consecutive visits of the vehicle to a negative tree edge e , the vehicle carries more than k vertices. Let t be the number of traversals the vehicle made on e , we show that $t \leq 2 * OPT_e$. It is trivially true if $t = 1$, so in the following subcases, we assume $k \geq 2$.

Subcase 1: t is odd. Assume that for every two consecutive visits, ~~but~~ except the last one, the vehicle dumps $k + 1$ pickup vertices to e in total. This is also the worst case our algorithm could have on e . Then $OPT_e \geq \frac{t-1}{2} + 1 = \frac{t+1}{2}$, it is equivalent to that $t \leq 2 * OPT_e - 1$.

Subcase 2: t is even. Assume that for every two consecutive visits, but except the last two, the vehicle dumps $k + 1$ pickup vertices to e in total. Then $OPT_e \geq \frac{t-2}{2} + 1 = \frac{t}{2}$, it is equivalent to that $t \leq 2 * OPT_e$.

Because each positive edge becomes negative in G' , in the two solutions, e is visited at most thrice. □

It is possible to further improve the ratio to $\frac{3}{2} - \frac{1}{2k}$ for the k -delivery TSP in trees of height 2. In this method, the negative edges are also split into pseudo edges which with load no more than k , and these pseudo edges are sorted in non-decreasing order of their edge costs in L_- . When the vehicle starts to dump its load to a negative edge of L_- , it is known that the vehicle load l_1 plus the load l_2 of the next positive pseudo edge is more than k . The algorithm chooses to dump only $k - l_1 - l_2$ vertices to the head of L_- , then the vehicle would pick up the l_2 vertices from the next positive edge, and dump the k vertices in compartments to the tail of L_- . This process is continued until all the vertices are served.

If a pseudo edge e of L_- is traversed n times, where $1 < n \leq k$, then for each of the first

$n - 1$ visits, there must be an edge in L_- traversing optimally. The last visit is for e itself, thus in every k visits, the vehicle needs to traverse at least $\lceil \frac{k-1}{2} \rceil + 1$ times in the optimal solution. Thus the solution with the smaller cost after applying this algorithm on G and G' is a $(\frac{3}{2} - \frac{1}{2k})$ -approximation for the k -delivery TSP in trees of height 2.



2.5.4 Pickup procedure for the Half-Load algorithm

Assume L_+ and L_- contain the edges in the first level of the transformed tree, then our solution can be expressed as *come_back3*(L_+, L_-). The Half-Load algorithm would be complete if we have appropriate pickup and deliver procedures.

Both our pickup and deliver procedures run recursively. Given a positive pseudo edge $e = (p(u), u)$, the pickup procedure is applied when the vehicle with a load α tries to cross e to pick up $n(e)$ items from T_u . It is also assumed that the actual route has already been built for the α vertices in the vehicle. Let S be the set of edges involved in generating e , and let the edges of S induce subtree T_S . Then the vehicle should service all the delivery vertices of T_S (by calling *come_back3*) and leave e (from u to $p(u)$) with $\alpha + n(e)$ items. A new edge e' is created and becomes the head of L_+ in this procedure (line 6 and 7 of Figure 2.17), just because the items in the vehicle should firstly be consumed.

The route implied by the planning phase for T_S assumes zero initial vehicle load. One may doubt whether the tour is still feasible if the vehicle already has a load. Recall that in the *come_back3* procedure, further picking up through a positive pseudo edge e is allowed, only if the load of e fits the rest of the vehicle capacity. Therefore when the algorithm decides to pick up the $n(e)$ items, it is always guaranteed that the vehicle can service all the vertices in S and come back to $p(u)$ without breaking the capacity constraint. This also shows the effectiveness of our planning phase. An example of picking up vertices from a pseudo edge is given in Figure 2.18.

The example in Figure 2.18 is a subgraph of the example in figure 2.14. We want to show how to pick up 6 vertices from the first positive pseudo edge $e_1 = (\text{speech bubble}, u)$ in figure 2.14. Before visiting e_1 , the vehicle is assumed to already carry 2 vertices, since $k = 6$, the vehicle should be able to leave $p(u)$ with 8 vertices. We only consider the schedule among the edges from u to its children involved in generating e_1 . Let $e_1^+, e_2^+, e_3^+, e_1^-$ and e_2^- be the 5 edges from u to its children as shown in figure 2.17, respectively from left to right. In this

procedure pickup(α, e)

Input: positive pseudo edge $e = (p(u), u)$; the vehicle load α satisfies $\alpha + n(e) \leq k$.

Output: routes showing how to pick up the $n(e)$ vertices from e .

```

1  if  $u$  is a leaf then
2      pickup  $u$  and return
3  endif
4
5  Comment: edges in  $L_+$  and  $L_-$  involved in generating  $e$  and
6  they are from  $u$  to some children of  $u$ 
7   $e' \leftarrow$  create a new pseudo edge for the vertices in the vehicle
8   $L_+ \leftarrow \{e'\} + L_+$ 
9
10 Comment:  $e'$  becomes the new head of  $L_+$ 
11  $\alpha, L_+ \leftarrow \text{come\_back3}(\text{merge}(L_+), L_-)$ 
12 for each edge  $e''$  of  $L_+$ 
13     Comment: pick up the rest vertices
14     pickup( $\alpha, e''$ )
15 endfor
17
18 return the route

```

Figure 2.17: Pickup procedure for the Half-Load algorithm

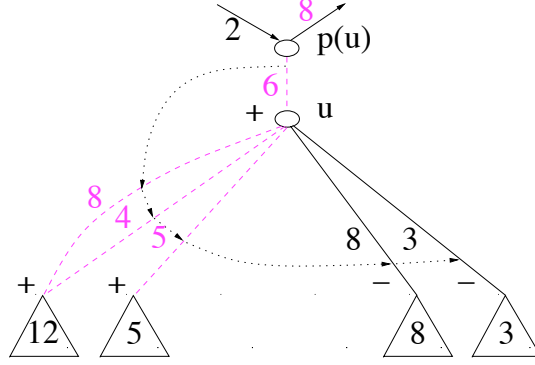


Figure 2.18: An example of picking up vertices from an edge $e = (p(u), u)$. $k = 8$ and the vehicle has an initial load of 2.

example, the vehicle visits these edges in the order of e_2^+ , e_1^- , e_1^+ , e_1^- , e_2^- , e_3^+ . The 8 vertices leaving $p(u)$ with the vehicle are from e_1^+ and e_3^+ .

2.5.5 Deliver procedure for the Half-Load algorithm

The deliver procedure is applied when the vehicle with α items tries to service a negative edge $e = (p(u), u)$. The pseudo codes of the deliver procedure are given in Figure 2.19.

The deliver function is similar to the pickup function, with the major difference that it needs an additional parameter L'_+ . In the algorithm, L'_+ is maintained to contain some positive pseudo edges (not in T_u), from which $n(u)$ delivery vertices of T_u can be served. We call our algorithm the Half-Load algorithm for two reasons. One reason is that when a negative edge e is being serviced, according to the *come back* rule, the vehicle should take more than k vertices during two consecutive visits to e . It can be viewed as that the vehicle is at least half full whenever it visits e . The other reason is that, when the vehicle comes back to u to pick up more items, say from an edge e' , the deliver procedure guarantees that the vehicle carries more than $\frac{k}{2}$ distinct items on its way from e' to e (except possibly one of the visits).

For the first reason, before servicing the delivery vertices of T_u , the edges of L'_+ are attached to the end of the list L_+ which consists of the positive pseudo edges from u to its children. According to the *come back* rule, before visiting a negative edge e' of L_- , e.g.,

procedure deliver(L'_+, α, e)

Input  vertices in the vehicle need to be delivered to $e = (p(u), u)$; L'_+ is a list of pseudo edges not in T_u

Output: routes showing how the vertices in the vehicle are delivered to e

```

1 if  $u$  is a leaf then
2     service  $u$  and return
3 endif
4
5 Comment: edges in  $L_+$  and  $L_-$  are from  $u$  to its children
6  $e' \leftarrow$  create a new pseudo edge for the vertices in the vehicle
7  $L_+ \leftarrow \{e\} + L_+ + L'_+$ 
8
9 return come_back3(merge( $L_+$ ),  $L_-$ )
```

Figure 2.19: Deliver procedure for the Half-Load  algorithm

When the vehicle gathers the vertices from the last positive pseudo edge of the original L_+ , the deliver procedure requires the information of the next positive pseudo edge of L'_+ to decide whether the vehicle should come back and pick up some more supplies from outside T_u , to make sure e' is traversed no more than twice of its optimum. Besides the first edge of L'_+ , if some delivery vertices of T_u are still not serviced, more edges of L'_+ are further needed for T_u .

An example showing the necessity of stitching L'_+ and L_+ is given in Figure 2.20. In this example, it is easy to see that all the positive pseudo edges are needed for serving the negative edges incident on u .

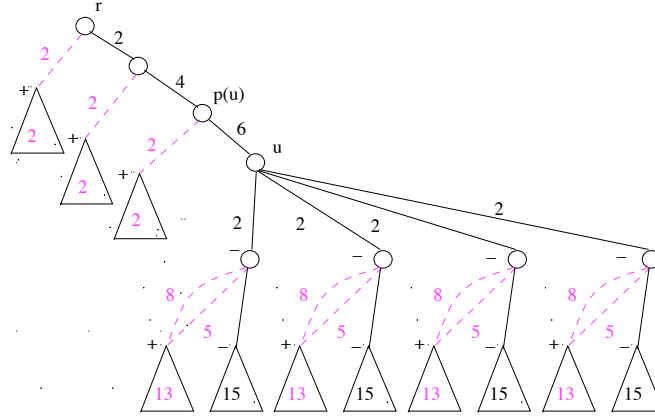


Figure 2.20: an example of delivering vertices to an edge $e = (p(u), u)$. $k = 8$ and r is the root of the tree.

For the second reason, after the stitching of L'_+ and L_+ , the algorithm merges two positive pseudo edges if they both have loads no more than $\lceil \frac{k}{2} \rceil$. Thus it is maintained that, when the procedure `come-back3` is called, L_+ only contains at most one pseudo edge with load of no more than $\lceil \frac{k}{2} \rceil$. Since the algorithm runs recursively, the priority of merging is given to positive pseudo edges in higher levels of the tree. The merging and also the way of merging in the algorithm are important for the performance guarantee. They together ensure that if the vehicle comes back and picks up some more vertices before serving a negative edge e' , then not only e' , but also the other edges along the path are traversed no more than twice of their optimum. A formal proof for the correctness of the strategy is shown in Lemma 2.5.3.

Lemma 2.5.3. *For two consecutive visits on a negative tree edge $e = (p(u), u)$ in the direction of $p(u) \rightarrow u$, the vehicle carries more than k vertices from outside T_u .*

Proof. In the algorithm after stitching L'_+ and L_+ , the new list L_+ includes all the available positive pseudo edges (from outside T_u) which can be used to service e . In addition to the proof of Lemma 2.5.2, in trees of general heights, the crossing on e from $p(u)$ to u might be triggered by insufficient load of the vehicle when it tries to service a negative edge inside T_u . In this case the vehicle would come back and cross e from u to $p(u)$ to pick up more items from outside T_u . We claim that the pickup vertices (not in T_u) passing through e from $p(u)$ to u , are picked up consecutively from the edges of the list L'_+ starting from its head, and all except possibly one of the edges in L'_+ contain more than $\lceil \frac{k}{2} \rceil$ vertices.

The first part of the claim holds, because in the delivery procedure, the edges of L'_+ is attached to the end of L_+ . So after all the positive edges in the original L_+ are consumed, the pickup vertices (not in T_u) crossing e from $p(u)$ to u , must be picked up consecutively from the edges of the list L'_+ . The latter part of the claim holds, because the delivery procedure gives merging priority to higher level pseudo edges of the tree. So no matter L'_+ is obtained during the planning phase, or during an earlier call of the pickup or delivery procedure, the merging step has always been deployed on L'_+ . This completes the proof. $\square$

While the proof follows naturally from the algorithm, in the following we further explain why the way of merging in the delivery function is important for the performance guarantee. Suppose there is an edge in L'_+ , say e' , has a load of no more than $\lceil \frac{k}{2} \rceil$. When the delivery function is called for T_u . This edge might be merged with another edge in L_+ to form a new pseudo edge containing more than $\lceil \frac{k}{2} \rceil$ vertices. When this new edge is requested, the vehicle will cross e from u to $p(u)$ to pick up the vertices in e' . For this traversal on e , the vehicle contains ~~only~~ no more than $\lceil \frac{k}{2} \rceil$ vertices from outside T_u . However, it is the only exception, since all other positive pseudo edges of L'_+ contains more than $\lceil \frac{k}{2} \rceil$ vertices. These edges may also be created by merging some pseudo edges with load more than $\lceil \frac{k}{2} \rceil$, but since the merging priority is given to pseudo edges in higher levels of the tree, they must have been formed and stayed in L'_+ when the delivery function is called for T_u . Thus the vertices in one such edge are all from outside T_u , and they will cross e with the vehicle all at one time. Note that for the last visit on e , although the vehicle may contain more than $\lceil \frac{k}{2} \rceil$ vertices, it might dump less than $\lceil \frac{k}{2} \rceil$ vertices on e .

Consider the tree in Figure 2.20. Let the 4 negative children of u be v_1, v_2, v_3 and v_4 respectively (from left to right). Before serving each negative edge $e_i = (u, v_i)$, for all $1 \leq i \leq k$, the first three positive pseudo edges in L_+ have loads 8, 5, 2 if merging is not adopted. After the pickup vertices of the first two pseudo edges are served, the vehicle has a load of 5, thus only 2 more vertices outside T_u can be picked up by the vehicle to serve the load of the negative edge. It is easy to see that in this example, the edge $e = (p(u), u)$ is traversed 3 times of its optimum. If the priority of merging is not given to edges in higher levels of a tree, e.g., if each positive pseudo edge with load 5 is merged with a positive edge with load 2, then the route remains the same as without merging.

The Half-Load algorithm alone is a 2-approximation for the k -delivery TSP in trees. The details of the performance guarantee for the Half-Load algorithm are shown in the proof of Lemma 2.5.4. They are important for the analysis of our final theorem.

Lemma 2.5.4. *The Half-Load algorithm is a 2-approximation for the k -delivery TSP in trees.*

Proof. It is not difficult to see that the tour is feasible, since in the tour all the vertices are served and the capacity constraint is always obeyed. Given an edge e of the tree, we prove that the number of traversals the Half-Load algorithm made on $e = (p(u), u)$ is no more than $2 * LB_e$. Let r be the residual of e ($n(u) \bmod k$). We have the following two cases:

Case 1: T_u is positive. Let the list L_+ contain all the pseudo edges on e . Each pseudo edge e' in L_+ is traversed only once in the algorithm, because the algorithm will not traverse this edge if the load of the vehicle plus the load of e' is more than the capacity. Thus the number of traversals the algorithm made on e is just the number of pseudo edges in L_+ . According to our merging rule in the planning phase, all pseudo edges possibly except one of them must have loads more than $\frac{k}{2}$. Assume the vehicle carries exactly $\frac{k}{2} + 1$ vertices each time during the first $|L_+| - 1$ visits on e , and let the last edge in L_+ be e' . This is the worst case our algorithm could have on e . We show that even in this case, $|L_+| \leq 2 * LB_e - 1$.

Subcase 1: $|L_+|$ is odd. For the first $|L_+| - 1$ visits, the lower bound of the number of traversals on e (part of LB_e) is $\frac{|L_+|-1}{2}$, if excluding $|L_+| - 1$ vertices from these visits. There must be one additional visit for these excluded vertices and the vertices in e' , e.g., if the number of these vertices equals to the residual of e . So in total $LB_e \geq \frac{|L_+|-1}{2} + 1 = \frac{|L_+|+1}{2}$, equivalent to $|L_+| \leq 2 * LB_e - 1$.

Subcase 2: $|L_+|$ is even. For the first $|L_+| - 2$ visits, if excluding $|L_+| - 2$ vertices, the lower bound of the number of traversals on e (part of LB_e) is $\frac{|L_+|-2}{2}$. The vehicle must also carry more than k vertices during the last two visits, so there must also be additional two visits in LB_e . Thus in total $LB_e \geq \frac{|L_+|-2}{2} + 2 = \frac{|L_+|+2}{2}$ equivalent to $|L_+| \leq 2*LB_e - 2$.

Case 2: T_u is negative. Unlike the first case, in this case we may have two visits on e , where the vehicle dumps less than $\frac{k}{2}$ pickup vertices to e . However, according to Lemma 2.5.3, the total vehicle load is more than k during every two consecutive traversals on e . The proof then follows much in the same way as the proof of Lemma 2.5.2.

□

2.5.6 The $\frac{5}{3}$ -approximation for the k -delivery TSP in trees of arbitrary heights

In this section, we prove that the smallest cost tour from three tours, namely obtained by applying the Full-Load algorithm on G (or G'), and the Half-Load algorithm on G and G' , is bounded by $\frac{5}{3}$ times of the optimum.

We explain the algorithm in [3] (which we call the Full-Load algorithm) as follows. For a vertex u , let $L_+ = \{c_1^+, c_2^+, \dots, c_i^+\}$ and $L_- = \{c_{i+1}^-, \dots, c_l^-\}$ be two lists consisting of the positive and negative children of u respectively. We also assume that in L_+ and L_- the vertices and the subtrees rooted on these vertices are sorted arbitrarily. In the Full-Load algorithm, the vehicle would pick up k items at a time *consecutively* from the subtrees $T_{c_1^+}, \dots, T_{c_i^+}$, and deliver the k items *consecutively* to the subtrees $T_{c_{i+1}^-}, \dots, T_{c_l^-}$. This also means that the vehicle services the subtrees in the sorted order, and when the vehicle starts to pick up (deliver), the vehicle would continue to load (consume) items if possible. For the sake of completeness, we show our implementation of the Full-Load algorithm in the appendix.

The Full-Load algorithm has its advantages and disadvantages. The advantage is that after the vehicle gathers k items, it traverses several subsequent edges optimally. The disadvantage is that, before the vehicle is full, and after delivering these k vertices begins, some edges of the tree might have to be traversed twice of their optimum. It is not difficult to see that the Half-Load algorithm exchanges its advantages and disadvantages with the Full-Load algorithm. This explains intuitively why we balance three tours as our final solution.

We give our analysis of the approximation ratio in Lemma 2.5.5.

Lemma 2.5.5. *In the solution produced by the Full-Load algorithm, each edge $e = (p(u), u)$ is traversed at most $LB_e + 1$ times.*

Proof. Consider the case when e is positive. In the Full-Load algorithm, when the vehicle starts to visit an edge, it will traverse the edge continuously for picking up vertices until all the load of this edge is serviced. Problems may occur if the vehicle has a load before its first visit on e . However, in this case the vehicle can carry less than k vertices only during its first and last visits on e . For all other visits, the vehicle picks up exactly k vertices from e . Thus in the solution e is visited at most $LB_e + 1$ times. The other case can be proved similarly. $\square$

We formally prove the approximation ratio $\frac{5}{3}$ of our final solution in Theorem 2.5.6. This theorem also implies that the time and space complexities of the Half-Load algorithm are $O(\lceil \frac{n^2}{k} \rceil)$.

Theorem 2.5.6. *The solution with the smallest cost from 3 tours, namely after applying the Full-Load algorithm on G (or G'), and applying the Half-Load algorithm on both G and G' , is a $\frac{5}{3}$ -approximation for the k -delivery TSP in trees of general heights.*

Proof. Given an edge e of the tree, we prove that the number of traversals in the 3 tours together made on e is no more than 5 times of its optimum. Let Sol_1 be the solution after applying the Full-Load algorithm on G (or G'), and Sol_2 and Sol_3 be the  after applying the Half-Load algorithm on G and G' respectively. As shown in Lemma 2.5.5, e is traversed at most $LB_e + 1$ times in Sol_1 . For Sol_2 and Sol_3 , we have the following cases:

Case 1: e is positive. If e is positive, e is visited at most $2 * LB_e - 1$ times in Sol_2 according to Lemma 2.5.4. e is negative in G' , so e is crossed at most $2 * LB_e$ times in Sol_3 (established in the proof of Lemma 2.5.2). We have proved that (in the proof of Case 1 in Lemma 2.5.4) in Sol_2 , e is traversed at most $2 * LB_e - 1$ times. Summing together, e is traversed at most 3 times LB_e in Sol_1 and Sol_2 , and therefore at most 5 times LB_e in the three solutions.

Case 2: e is negative. Because of the symmetry, if e is negative in G , its corresponding edge e' is positive in G' , so similarly we can prove that the total number of traversals in Sol_1 and Sol_3 is bounded by thrice of LB_e . Thus we have proved that e is traversed at most $5 * LB_e$ times in the 3 solutions. $\square$

2.6 A $(6\sqrt{k})$ -approximation for the capacitated Dial-a-Ride problem on height balanced trees

For the capacitated Dial-a-Ride problem with arbitrary capacity in *general graphs*, the best known ratio $O(\sqrt{k} \log n)$ is due to M. Charikar and B. Raghavachari [49]. In fact, their solution is for a type of special structured trees, which they call height-balanced trees [49].

A height-balanced tree is a rooted weighted tree such that: (1) the distance from the root to any leaf is the same for all leaves. (2) for all leaves, the path from the root to the leaf consists of the same number m of edges. (3) for $i = 1, \dots, m$, define level i to be all the edges which are the i th edge on the (shortest) path from the root to some leaf. Then edges in the same level have the same length. It is shown in [49] that an algorithm with ratio α for any problem in height-balanced trees is a $(\alpha \log n)$ -approximation for the same problem in general graphs.

For two edges e and e' at level l of the height-balanced tree, define an (e, e') -item to be an item that needs to be transported from e to e' . Let $y(e, e')$ be the number of all the (e, e') -items. In [49] it is established that the optimum of the capacitated Dial-a-Ride problem in height balanced trees is at least $\frac{1}{\sqrt{k}} \sum_{e, e' \in \text{level } l} \lceil \frac{y_{e, e'}}{k} \rceil$. This is called *the wait lower bound* in [49].

The algorithm in [49] is a $8\sqrt{k}$ -approximation for the capacitated Dial-a-Ride problem in height balanced trees. It works as follows. For each item x , let $p(x)$ be the path from the source of x to the destination of x . Let the turning point of x be the highest vertex in the height-balanced tree on $p(x)$, and let $S(v)$ be the set of all items whose turning point is v . The algorithm in [49] visits all the turning points in depth-first order, and when a turning point v is visited, all the items in $S(v)$ are picked up and delivered to their destinations.

For each item x in $S(v)$, $p(x)$ has exactly two edges, one upward and the other downward, in each level of the height-balanced tree. Assuming that v and x are in level m and $m + h$ of the tree respectively, x is associated with an edge sequence $E(x) = (e_{m+1}, e_{m+2}, \dots, e_{m+h}, e'_{m+1}, e'_{m+2}, \dots, e'_{m+h})$, where e_{m+i} is the upward level- $(m + i)$ edge in $p(x)$ and e'_{m+i} is the downward level- $(m + i)$ edge in $p(x)$. In other words, the edges in $E(x)$ are sorted in the order of increasing level, and within the same level, the upward edge is placed before the downward edge.

Given an arbitrary order of the edges in the tree, the algorithm in [49] sorts the items

in $S(v)$ in lexicographic order of their edge sequences. Group every k items according to this sorted order. For each group, the algorithm picks up the k items from their sources in a depth-first order, and delivers these k items to their destinations also in a depth-first order. It is not difficult to prove that all the (e, e') items occur consecutively in this order.  an argument similar  the proof of Lemma 2.5.5, the number of groups that contains some (e, e') items is at most $1 + \lceil \frac{y_{e,e'}}{k} \rceil \leq 2\lceil \frac{y_{e,e'}}{k} \rceil$. Since a depth first search is performed for each group, the total number of times level- l edges are traversed is at most $2 \sum_{e,e' \in \text{level } l} (1 + \lceil \frac{y_{e,e'}}{k} \rceil + 1 + \lceil \frac{y_{e',e}}{k} \rceil) \leq 8 \sum_{e,e' \in \text{level } l} \lceil \frac{y_{e,e'}}{k} \rceil$. Thus the algorithm in [49] gives an $8\sqrt{k}$ -approximation for the capacitated Dial-a-Ride problem in height-balanced trees.

The Half-Load strategy can be applied for the capacitated Dial-a-Ride problem in height-balanced trees. Our algorithm also works on the list of all the items sorted as above. It scans the items in the sorted order, and maintains a list $L(e, e')$ consisting of groups of (e, e') -items, for two edges e and e' at the same level of the height balanced tree. Moreover each group in $L(e, e')$ (except possibly one) has more than $\lceil \frac{k}{2} \rceil$ items.  maintains two variables  x_1 and x_2 pointing to the current and next items respectively. Let the sources (destinations) of x_1 and x_2 be u_s (u_d) and v_s (v_d) respectively. Define $\text{lca}(u, v)$ to be the least common ancestor of vertices u and v . Let $u_1 = \text{lca}(u_s, v_s)$ and $u_2 = \text{lca}(u_d, v_d)$. To ease the description we add a pseudo vertex $p(r)$ as the parent of the root r . Consider the following cases:

Case 1: e and e' are not initialized. We set e to be $(p(u_1), u_1)$ and e' to be $(p(u_2), u_2)$. x_1 and x_2 are added to the last group of $L(e, e')$. If the last group of $L(e, e')$ already has k items, we attach a new group containing x_1 and x_2 to the tail of $L(e, e')$.

Case 2: $e = (p(u_1), u_1)$ and $e' = (p(u_2), u_2)$. In this case we add x_2 to the last group of $L(e, e')$. If the last group of $L(e, e')$ already has k items, we attach a new group containing x_2 to the tail of $L(e, e')$.

Case 3: $e \neq (p(u_1), u_1)$ or $e' \neq (p(u_2), u_2)$. In this case we merge two groups of $L(e, e')$ with no more than $\lceil \frac{k}{2} \rceil$ items as long as such  groups exist. We add the groups in $L(e, e')$ to $L((p(u_1), u_1), (p(u_2), u_2)))$. We then set e to $(p(u_1), u_1)$ and e' to $(p(u_2), u_2)$, and a new group containing x_2 is created for $L(e, e')$.

As the last step, the vehicle performs a depth-first search to service the items in each group. Since it is maintained that there is at most one group with less than or equal to $\lceil \frac{k}{2} \rceil$ items in each list $L(e, e')$, the number of groups that contain some (e, e') -item is at most $2\lceil \frac{y_{e,e'}}{k} \rceil - 1$. Therefore in the solution with the smaller cost from the two solutions produced

by the algorithm in [49] and the Half-Load algorithm, the total number of times level- l edges are traversed is at most $2 \sum_{e,e' \in \text{level } l} ((1 + \lceil \frac{y_{e,e'}}{k} \rceil + 2 \lceil \frac{y_{e',e}}{k} \rceil - 1)/2 + (1 + \lceil \frac{y_{e',e}}{k} \rceil + 2 \lceil \frac{y_{e,e'}}{k} \rceil - 1)/2) = 6 \sum_{e,e' \in \text{level } l} \lceil \frac{y_{e,e'}}{k} \rceil$. So we establish the following theorem.

Theorem 2.6.1. *The solution with the smaller cost from the two solutions produced by the algorithm in [49] and the Half-Load algorithm, is a $6\sqrt{k}$ -approximation for the capacitated Dial-a-Ride problem in height balanced trees.*

Proof. Omitted. □